



LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS C++

Herança

Conteúdo da Aula

- Herança

Herança

- Herança é um dos pontos chave de programação orientada a objetos (POO). Ela fornece meios de promover a extensibilidade do código, a reutilização e uma maior coerência lógica no modelo de implementação. Estas características nos possibilitam diversas vantagens, principalmente quando o mantemos bibliotecas para uso futuro de determinados recursos que usamos com muita frequência.
- Uma classe de objetos "veículo", por exemplo, contém todas as características inerentes aos veículos, como: combustível, autonomia, velocidade máxima, etc. Agora podemos dizer que "carro" é uma classe que têm as características básicas da classe "veículo" mais as suas características particulares. Analisando esse fato, podemos concluir que poderíamos apenas definir em "carro" suas características e usar "veículo" de alguma forma que pudéssemos lidar com as características básicas. Este meio chama-se herança.
- Agora podemos definir outros tipos de veículos como: moto, caminhão, trator, helicóptero, etc, sem ter que reescrever a parte que está na classe "veículo". Para isso define-se a classe "veículo" com suas características e depois cria-se classes específicas para cada veículo em particular, declarando-se o parentesco neste instante.
- Outro exemplo: Imagine que já exista uma classe que defina o comportamento de um dado objeto da vida real, por exemplo, animal. Uma vez que eu sei que o leão é um animal, o que se deve fazer é aproveitar a classe animal e fazer com que a classe leão derive (herde) da classe animal as características e comportamentos que a mesma deve apresentar, que são próprios dos indivíduos classificados como animais.
- Ou seja, herança acontece quando duas classes são próximas, têm características mútuas mas não são iguais e existe uma especificação de uma delas. Portanto, em vez de escrever todo o código novamente é possível poupar algum tempo e dizer que uma classe herda da outra e depois basta escrever o código para a especificação dos pontos necessários da classe derivada (classe que herdou). (WikiBooks)

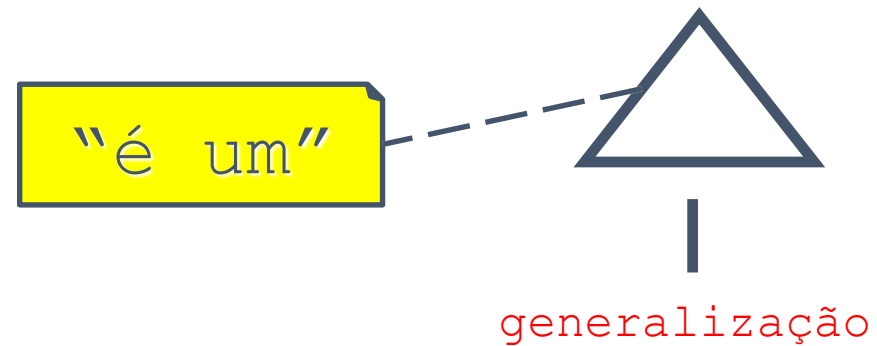
Herança

- Herança:
 - permite usar classes já definidas para derivar novas classes;
 - nova classe herda propriedades (dados e métodos) da classe base.
- Membro da classe derivada pode usar os membros públicos (public) e protegidos (protected) da sua classe base (como se fossem declarados na própria classe)
- Membro protegido (protected) funciona como:
 - membro público para as classes derivadas;
 - membro privado para as restantes classes.

Herança

- Com o mecanismo de herança, classes podem ser desenvolvidas incrementalmente a partir de classes básicas simples.
- Cada vez que se deriva uma nova classe, a partir de uma já existente, pode-se herdar algumas ou todas as características da classe pai (básica).
- Essas características podem ser traduzidas por flexibilidade para o programador.

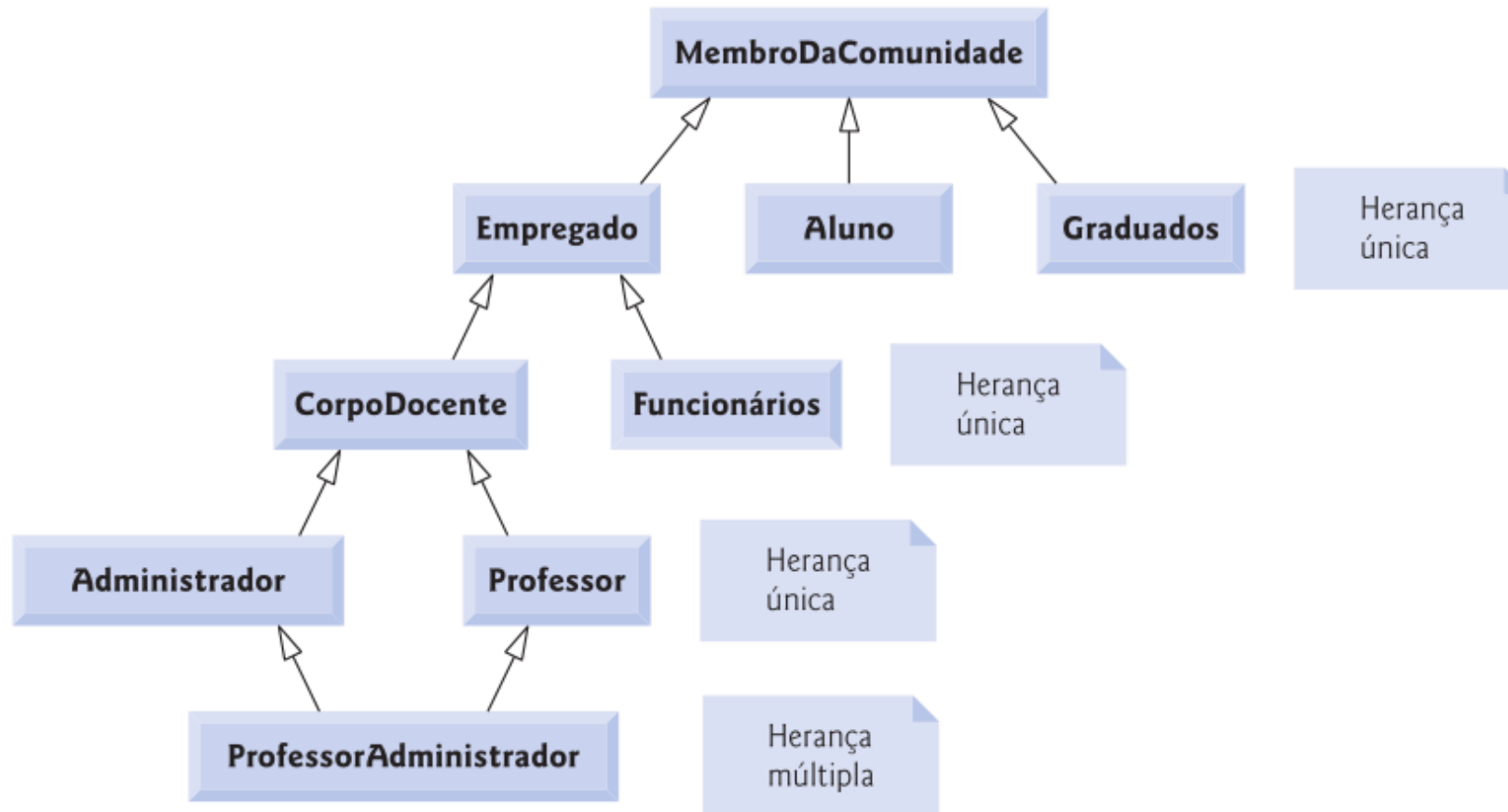
Herança: Generalização

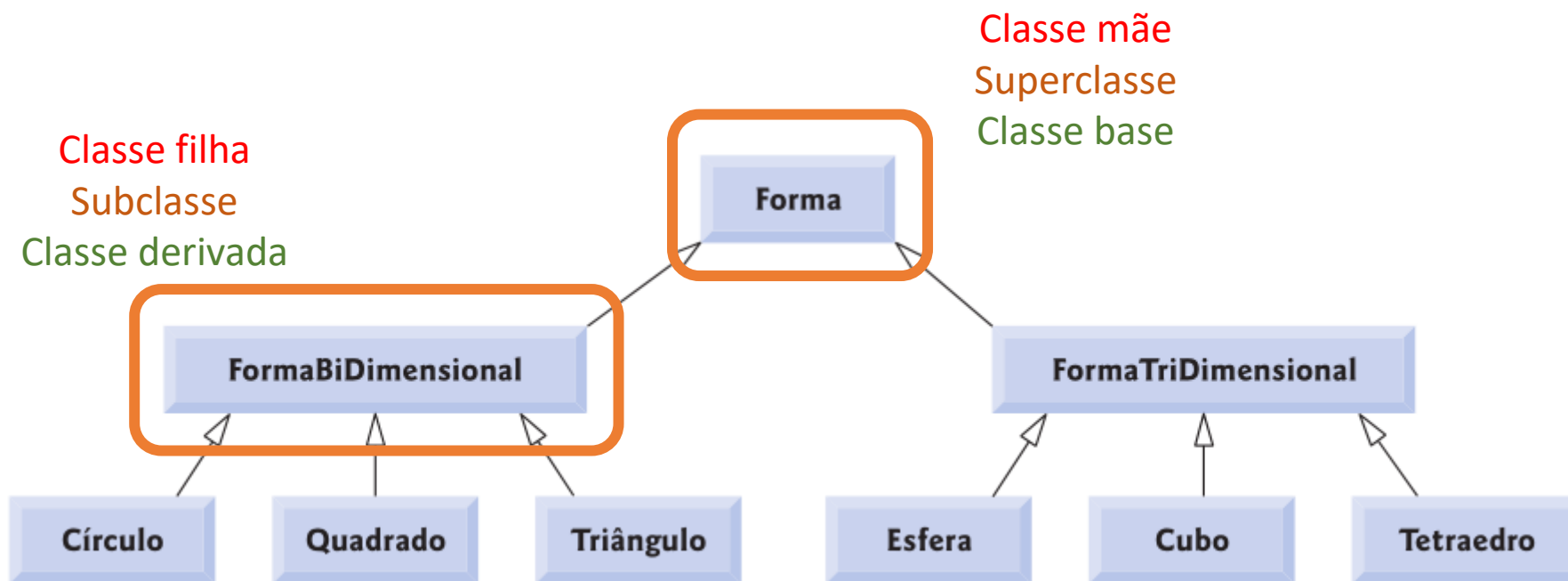


Hierarquia - Generalização

Classe básica	Classes derivadas
Aluno	AlunoDeGraduação , AlunoDePósGraduação
Forma	Círculo , Triângulo , Retângulo , Esfera , Cubo
Financiamento	FinanciamentoDeCarro , FinanciamentoDeReformaDeCasa , FinanciamentoDeCasaPrópria
Empregado	CorpoDocente , Funcionários
Conta	ContaCorrente , ContaPoupança

Exemplo de Hierarquia



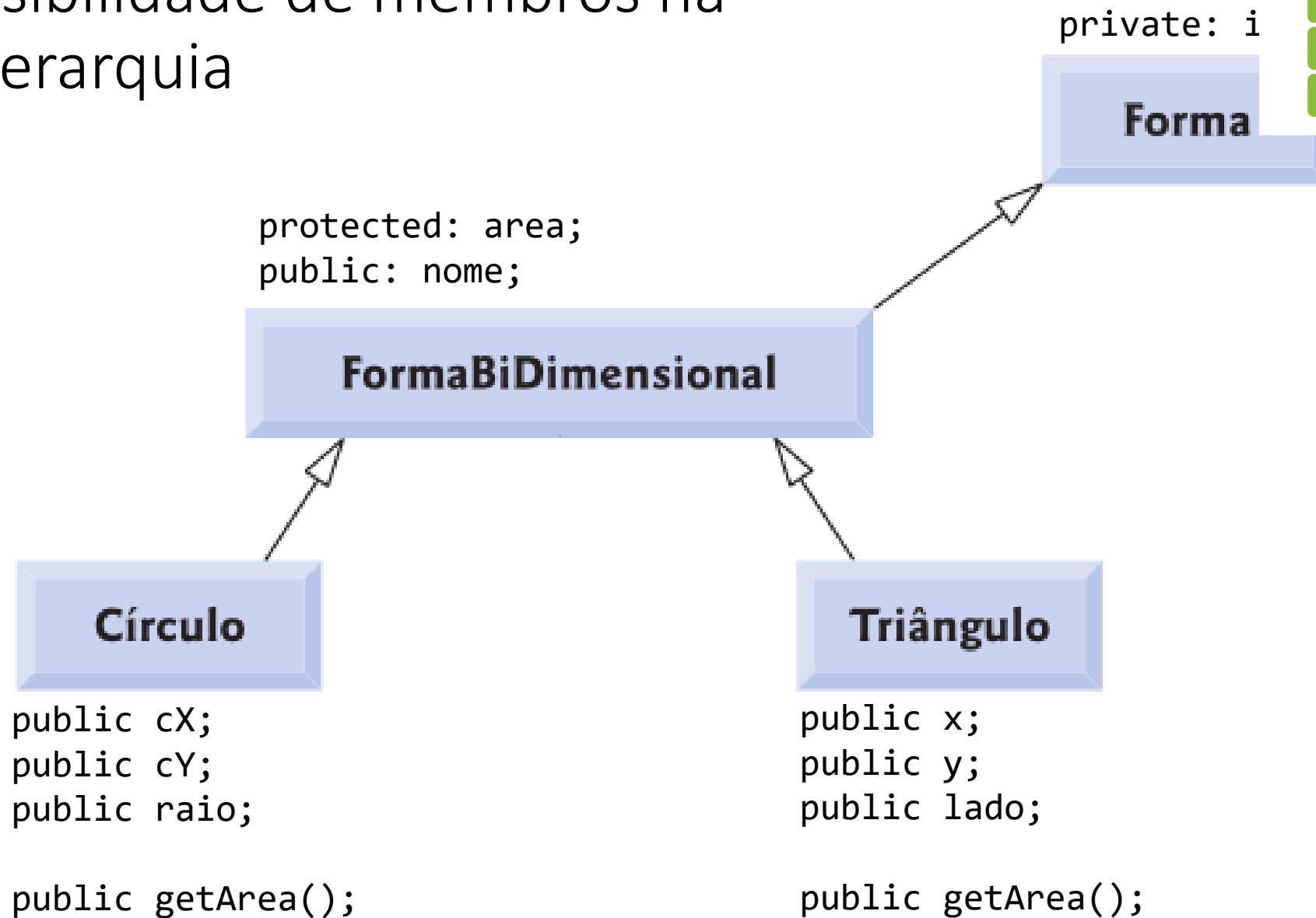


```
class FormaBiDimensional : public Forma
```

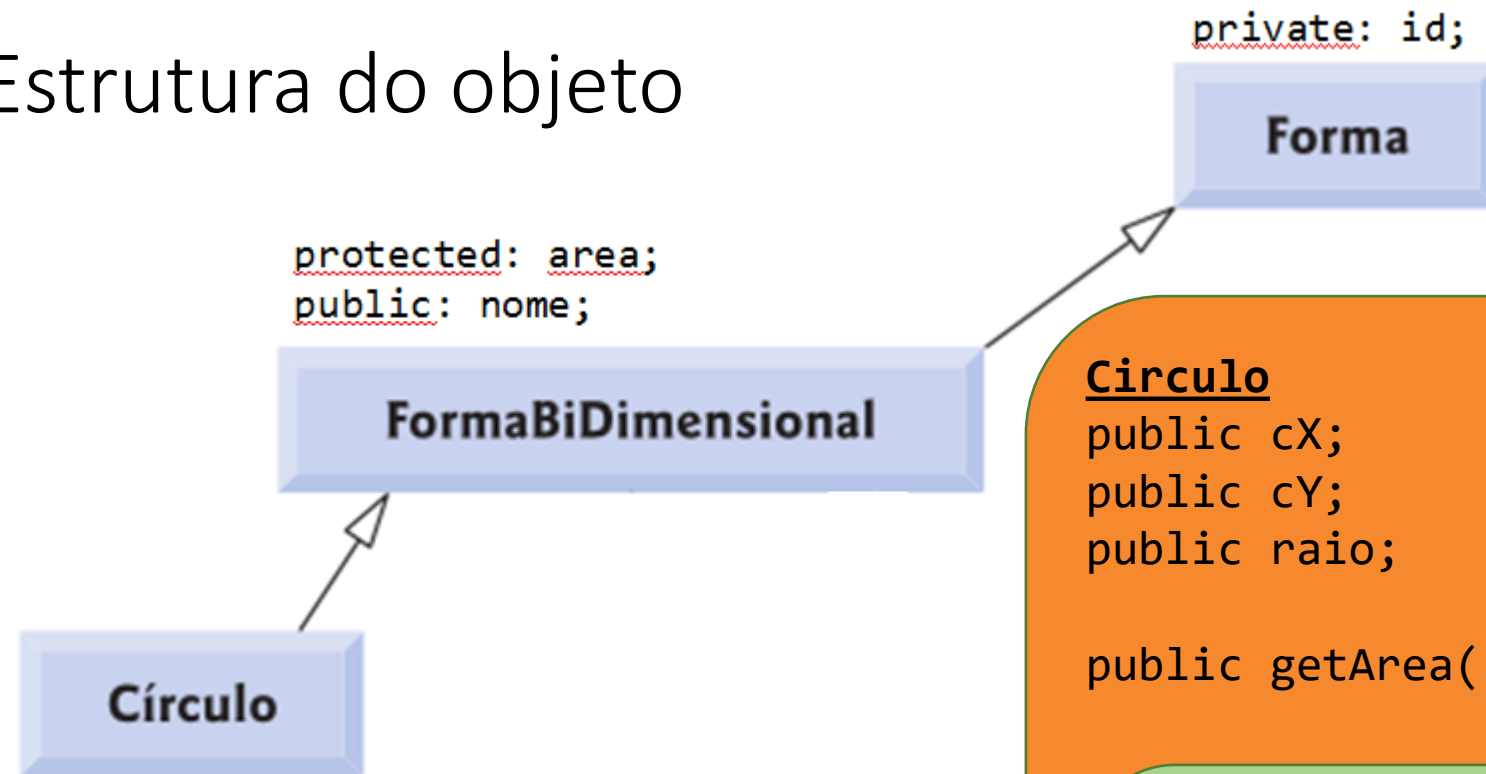
Classe filha
Subclasse
Classe derivada

Classe mãe
Superclasse
Classe base

Visibilidade de membros na hierarquia



Estrutura do objeto



```
public cX;  
public cY;  
public raio;
```

```
public getArea();
```

Círculo

```
public cX;  
public cY;  
public raio;
```

```
public getArea();
```

FormaBiDimensional

```
protected: area;  
public: nome;
```

Forma

```
private: id;
```

Quais os membros acessíveis em cada classe?

